

Factory Design pattern

18 September 2026 17:57

Absolutely. Let's understand **Factory Design Pattern from scratch**, in the same way we did Builder.

Factory Design Pattern

The basic idea

Factory Pattern = Don't make the client decide which object to create. Let a separate class decide.

The word **Factory** is literal here:

You ask the factory for something → the factory creates the appropriate object → gives it to you.

Interview definition

If an interviewer asks:

What is Factory Design Pattern?

You can say:

"Factory Pattern is a creational design pattern where object creation is delegated to a separate factory class. The client requests an object through a common interface without directly depending on the concrete implementation or its creation logic."

That's the formal answer.

Super-simple version:

Factory = give the object-creation responsibility to another class.

1. First, without Factory Pattern

Suppose we're building a payment system.

We have:

```

</> Java
class CreditCardPayment {
    void pay() {
        System.out.println("Paying using Credit Card");
    }
}

class UPIPayment {
    void pay() {
        System.out.println("Paying using UPI");
    }
}

class PayPalPayment {
    void pay() {
        System.out.println("Paying using PayPal");
    }
}
  
```

Now the client wants to make a payment.

```

</> Java
public class Main {
    public static void main(String[] args) {

        String paymentType = "UPI";

        if (paymentType.equals("UPI")) {
            UPIPayment payment = new UPIPayment();
            payment.pay();
        } else if (paymentType.equals("CREDIT_CARD")) {
            CreditCardPayment payment = new CreditCardPayment();
            payment.pay();
        } else if (paymentType.equals("PAYPAL")) {
            PayPalPayment payment = new PayPalPayment();
  
```

```

        Payment payment = new Payment();
        payment.pay();
    }
}

```

What's the problem?

The client (Main) knows too much.

It knows:

- UPIPayment
- CreditCardPayment
- PayPalPayment
- how to create each one
- which if/else corresponds to which class

Imagine we add:

```

// Java
class ApplePayPayment

```

Now we have to modify Main .

That's undesirable.

2. First improvement: Interface

We can notice that all payments have the same operation:

```

// Java
pay()

```

So let's create an interface.

```

// Java
interface Payment {
    void pay();
}

```

Now:

```

// Java
class UPIPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying using UPI");
    }
}

```

```

// Java
class CreditCardPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying using Credit Card");
    }
}

```

```

// Java
class PayPalPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying using PayPal");
    }
}

```

Now all three are Payment .

3. Now comes the Factory

We create a separate class whose job is:

Create the appropriate Payment object.

```

// Java
class PaymentFactory {

    public static Payment createPayment(String type) {

```

NOTICE THAT METHOD IS STATIC

```

    if (type.equals("UPI")) {
        return new UPIPayment();
    } else if (type.equals("CREDIT_CARD")) {
        return new CreditCardPayment();
    } else if (type.equals("PAYPAL")) {
        return new PayPalPayment();
    }

    throw new IllegalArgumentException("Invalid payment type");
}
}

```



Now look at what happened.

The ugly object-creation logic has moved from:

```

</> Java
Main

```

to:

```

</> Java
PaymentFactory

```

4. Client becomes very simple

```

</> Java
public class Main {

    public static void main(String[] args) {

        Payment payment =
            PaymentFactory.createPayment("UPI");

        payment.pay();
    }
}

```

Output:

```

Paying using UPI

```

The client doesn't need to know:

```

</> Java
new UPIPayment()

```

It simply says:

```

</> Java
PaymentFactory.createPayment("UPI");

```

★ Most important concept

This is the key line:

```

</> Java
Payment payment = PaymentFactory.createPayment("UPI");

```

Notice the left side:

```

</> Java
Payment payment

```

not:

```

</> Java
UPIPayment payment

```

Why?

Because the client shouldn't be tightly coupled to the concrete implementation.

The client only knows:

```

Payment

```

The Factory knows:

```

UPIPayment
CreditCardPayment
PayPalPayment

```

Factory is NOT mainly about avoiding new

This is a common interview misunderstanding.

Some people think:

"Factory Pattern means we don't use new."

✗ Not true.

The Factory itself uses new:

```
Java
return new UPIPayment();
```

The point is:

The client doesn't need to know where/how the concrete object is created.

Factory vs Builder

Since you just learned Builder, this distinction is important.

Factory

Answers:

"WHICH object should I create?"

Example:

```
Java
PaymentFactory.createPayment("UPI");
```

Could create:

```
UPIPayment
CreditCardPayment
PayPalPayment
```

Builder

Answers:

"HOW do I construct this particular object?"

Example:

```
Java
User user = new User.Builder()
    .setName("Supraja")
    .setAge(25)
    .setEmail("abc@gmail.com")
    .build();
```

The object type is already known:

```
User
```

We're just constructing it step-by-step.

Why should client not be tightly coupled with creation logic of objects?

1. The client should depend on the Payment abstraction/interface rather than concrete payment classes.

Example:

```
Payment payment = PaymentFactory.createPayment("UPI");
```

The client knows Payment, not UPIPayment.

- 2.

2. "Main method is supposed to make payment. doesn't need to know payment classes" —

✓ Correct

This is a very good point.

The main() method's responsibility is:

```
Java
payment.pay();
```

- 3.

It shouldn't need to worry about:

```
Java
```

```
new UPIPayment();
new CreditCardPayment();
new PayPalPayment();
```



So we move object-creation responsibility to the Factory.

You can say:

The client should focus on using the object, not on deciding which concrete object to create.

Summary: (these are my answers, asked gpt to review)

Your answer, interview-ready

I'd phrase your three points like this:

1. **Client should depend on the Payment abstraction rather than concrete payment classes.**
2. **The client/main method should focus on making the payment; it doesn't need to know how or which concrete payment object is created.**
3. **Separating object creation from business logic gives each class a clearer responsibility and supports the Single Responsibility Principle.**

why are we using interface? why cant we use abstract classes?

2. Using an abstract class

We could instead do:

```
Java
```

```
abstract class Payment {
    abstract void pay();
}
```

Then:

```
Java
```

```
class UPIPayment extends Payment {
    @Override
    void pay() {
        System.out.println("UPI payment");
    }
}

class CardPayment extends Payment {
    @Override
    void pay() {
        System.out.println("Card payment");
    }
}
```

And the Factory can still return:

```
Java
```

```
static Payment createPayment(String type)
```

So this works perfectly.

So why do we commonly use an interface?

1. We only need a contract

Our requirement is simply:

"Every payment must have a pay() method."

We don't necessarily need to provide **common implementation or state**.

That's exactly what an interface is good at.

```
interface Payment { void pay(); }
```

2. Different payment classes may have completely different implementations

UPI:

```
Java
```

```
class UPIPayment implements Payment {
    public void pay() {
        // UPI-specific logic
    }
}
```

Card:

```
Java
```

```
class CardPayment implements Payment {
    public void pay() {
        // Card-specific logic
    }
}
```

There may be very little common code between them.

So an interface makes sense.

3. Java allows only single class inheritance

This is an important reason.

Suppose:

```
Java
class UPIPayment extends Payment
```

Now `UPIPayment` cannot extend another class:

```
Java
class UPIPayment extends Payment, SomeOtherClass // ❌
```

But it can implement multiple interfaces:

```
Java
class UPIPayment implements Payment, Refundable, Auditable
```

So interfaces give more flexibility.

When would I use an abstract class instead?

Use an abstract class when your child classes have shared state or shared implementation.

For example:

```
Java
abstract class Payment {
    protected double amount;

    Payment(double amount) {
        this.amount = amount;
    }

    void printReceipt() {
        System.out.println("Receipt generated");
    }

    abstract void pay();
}
```

Now:

```
Java
class UPIPayment extends Payment {
    UPIPayment(double amount) {
        super(amount);
    }

    @Override
    void pay() {
        System.out.println("Paying ₹" + amount + " using UPI");
    }
}
```

Here an abstract class makes sense because all payments share:

```
amount
constructor
printReceipt()
```



The distinction

Think of it this way:

Interface

"What can you do?"

```
Java
```

Payment
↓
pay()

It primarily defines a contract.

Abstract class

"What are you, and what do you already have in common?"

```
Payment
├─ amount
├─ printReceipt()
└─ pay() ← abstract
```

It can define **common state + common behavior + abstract behavior**.

Therefore, for our Factory example:

We chose:

```

</> Java
interface Payment

```

because our main requirement was simply:

All payment types should follow the same contract, `pay()`.

BUTTTTTTTTTTTTTTTT

Factory Pattern doesn't care whether the abstraction is an interface or abstract class.

The important part is:

```

Client
  ↓
Payment ← abstraction
  ↑
  |
Factory creates concrete implementation
  ↓
UPIPayment / CardPayment / PayPalPayment

```

So don't memorize "Factory = interface."

Instead remember:

Factory = client depends on an abstraction, while the Factory handles creation of the concrete implementation.



why to use static method in factory class?

You **don't have to use a static method** in a Factory. It's a design choice.

Let's understand why we often see it.

1. Without static

Suppose your Factory is:

```

4) Java

class PaymentFactory {

    Payment createPayment(String type) {
        if (type.equals("UPI")) {
            return new UPIPayment();
        }
        return new CardPayment();
    }
}

```

To use it:

```
(7) Java
PaymentFactory factory = new PaymentFactory();
Payment payment = factory.createPayment("UPI");
```

Notice something strange?

We created:

```
Java
new PaymentFactory()
```

just to call a method whose job is **creating other objects**.

The Factory object itself doesn't contain any meaningful data.

2. With static

```
Java
class PaymentFactory {
    static Payment createPayment(String type) {
        if (type.equals("UPI")) {
            return new UPIPayment();
        }
        return new CardPayment();
    }
}
```

Now:

```
Java
Payment payment =
    PaymentFactory.createPayment("UPI");
```

No need to create a Factory object.

So the reasoning is:

Factory has no state

Imagine:

```
Java
PaymentFactory factory = new PaymentFactory();
```

What does this `factory` object contain?

```
factory
|
└ ???
```

Usually **nothing**.

The Factory is just providing a creation operation.

So we can make that operation static:

```
Java
PaymentFactory.createPayment(...)
```

instead of:

```
Java
new PaymentFactory().createPayment(...)
```

BUTTTTTTTTTTTTTTTTT

But here's the important part ⚠️

Static is NOT required for the Factory Pattern.

You can absolutely write:

```
Java
PaymentFactory factory = new PaymentFactory();

Payment payment = factory.createPayment("UPI");
```

and it's still a Factory.

When would you NOT use static?

Suppose the Factory itself needs some configuration/state:

```
Java
class PaymentFactory {
    private PaymentConfig config;

    PaymentFactory(PaymentConfig config) {
```

```
        this.config = config;
    }

    Payment createPayment(String type) {
        // use config
        ...
    }
}
```

Now you **need an instance** because different Factory objects could have different configurations.

For example:

```
Java
PaymentFactory factory =
    new PaymentFactory(indianPaymentConfig);
```



versus:

```
Java
PaymentFactory factory =
    new PaymentFactory(internationalPaymentConfig);
```

A static method wouldn't naturally work with instance-specific state.

Learn:

✅ "If the Factory doesn't need instance state, a static factory method can avoid creating a Factory object. But Factory Pattern itself does not require static."